Speaker notes — protoST — FAST 2026 (English)

Extracted from `deck.html` (the same content as the published deck). The PDFs in `slides/` carry no notes.

Slide 1 — protoST

Introduce yourself in one sentence. Say from the start that this is an experiment and that the talk is looking for curious people, not users.

Slide 2 — An experiment looking for enthusiasts.

Set the tone: no marketing. If something does not work, I say so. Everything shown has a test or a report behind it.

Slide 3 — Sound familiar?

Ask them as questions and let the audience answer. Do not state their pains for them. Do not promise that protoST solves them: it opens another way to try.

Slide 4 — protoCore: one core, several languages

Three properties, one example of each if asked. protoCore is a separate project; protoST is its demonstrator.

Slide 5 — A mutable object is a reference to an immutable snapshot

Modifying installs a new snapshot; whoever held the previous one still sees it whole. That is why passing objects between threads needs neither copies nor locks.

Slide 6 — Demo 1 · the code you already know

Run ./run_demo.sh 01-familiar-code.st. Show the file first. If it fails: scriptreplay of recordings/01. Point out that new sends initialize; the last line prints the total (270): a script shows only what it prints.

Slide 7 — Any object can be an actor

wait, thenDo:, whenAll:. An error inside the actor comes back, with its class, to whoever is waiting. An actor that is waiting does not handle other messages; if two actors wait on each other, protoST reports it as an error.

Slide 8 — The same computation, on one thread and on twelve actors

Run ./run_demo.sh 02-actors-parallelism.st. The last line (time speedup) is the speedup: state the one that appears on screen, not one from memory. It depends on the machine and its load: with protoCore 2.14.6, on the talk laptop, a median of 3.8× (3.3–4.2×), one thread at about 925 ms and twelve actors at about 245 ms. It is less than the 4.4× of before because 2.14.6 made the single-thread code 45 % faster (and the twelve actors 35 %): see hard-questions.md. The code of the computation did not change: what changed is who runs it.

Slide 9 — Worlds it could open up

Be explicit about the third one: it works when both runtimes are embedded in one program, and what is tested is one direction: protoScala reads and writes protoST objects. The reverse (protoST using another runtime's objects or modules) does not work yet, and the installed binaries do not load each other. docs/INTEROP.md §0 says exactly what works. Go on to demo 4.

Slide 10 — Another program delivers the data; protoST processes and publishes it

Run ./run_demo.sh 04-connected-twin.st. Show the code: the names are Pharo's (Stdio, asFileReference, writeStreamDo:). Another language delivers the data, protoST processes it with actors and exposes it over HTTP; none of it needed an image. If it fails: scriptreplay of recordings/04.

Slide 11 — Every benchmark verifies its result

The figures come from the reports; if they are not in a report, they are not said. The harness fails if a benchmark does not report the correct result: that way a crash never looks like an improvement. Every figure on the slide: report of 12/10 on protoCore 2.14.6 (the published protoST binary on 2.14.6 and on 2.14.5, medians of 14 interleaved runs with A/A pairs; CPython 3.14 on the same machine). The 14–20× range is honest: with small N the work is the difference of two similar times, and noise moves it; the whole process gives a geometric mean of 3.2–3.3×. protoCore 2.14.6 against 2.14.5: demo 2 45 % faster on one thread and 35 % with twelve actors, exceptions 7 %, the rest the same within the noise; under the fixed 640 MB ceiling, K5's peak falls from 1,165 to 690 MB. 0.6.2 against 0.6.1 (heap report of 12/10): actors 16–43 % faster, a fifth to a half of the memory, pauses of at most 0.27 ms; but peak memory is no longer capped (an allocating loop reached 0.88–3.5 GB).

Slide 12 — All written down and verified

Read the headlines. The message: we hide nothing; if something surprises you and it is not in chapter 14, it is a bug and we want to know about it.

Slide 13 — Adversarial reviews, before this talk

Tell the story of the intermittent hang as an example of method: instead of retrying, capture the stack of the hung process. The ≈1100 programs are the adversarial audit's (docs/STATUS.md); the tests are the ctest suite, a different thing. The I/O review (source: protoIO/docs/specs/2026-09-30-protoio-design.md; fixes in protoST commit c6beb6e) found, among others, that a child program that did not read its input killed the interpreter (SIGPIPE), that several actors reading standard input mixed lines, and a pool mutex held across an allocation. It also exposed a protoCore defect when creating threads, fixed in protoCore 2.6.1. 1101 is the suite of 0.6.5 with protoCore 2.14.6 on Linux (0.5.0 had 1068); CI runs 1100 on Linux and macOS and 1097 on Windows. In 0.6.3 a GCC 15 warning exposed a latent defect: the computed-goto dispatch did not release the collector's transient pins; fixed, with its test.

Slide 14 — Write a program the way you would write it.

Close with the concrete invitation and open the floor to questions. Keep hard-questions.md at hand. Installing is two .deb files (protoCore 2.14.6 and protoST 0.6.5) attached to the GitHub releases; on Windows, inside WSL2 with Ubuntu 24.04. The four demos come with the package, in /usr/share/protoST/talks/2026-10-15-fas/demos. The protoST 0.6.5 .deb requires protoCore 2.14.6; built from source it needs 2.11 or later, and it also builds natively on Windows; there is no published Windows package yet.

Slide 15 — Demo 3 · digital twin (module) · abridged output

Run ./run_demo.sh 03-digital-twin.st. The time per cycle is around 50 ms, not 150, because the three sensors answer in parallel. Walk through the code if there is time.

Slide 16 — Cells, snapshots and a GC that starts early

Optional 10-minute module. Draw the atomic snapshot on the whiteboard if there are questions. The collector is protoCore 2.14.6's, whose mark work list has a fixed size (8 MiB): under the fixed 640 MB ceiling, the peak of a 500,000-element array fell from 1,165 to 690 MB (report of 12/10 on 2.14.6). Since 0.6.2 protoST uses protoCore's adaptive heap controller (up to 0.6.1, a fixed 640 MB ceiling): actor programs 16–43 % faster with a fifth to a half of the memory; the price is that the peak is no longer capped (0.88–3.5 GB in an allocating loop; PROTOCORE_HEAP_LIMIT_CELLS fixes it). That needed a fix of protoST's own: each stop-the-world pause scanned a whole 1 M-slot frame array per thread (pauses of up to 20–31 ms with six to twelve workers); the array now grows on demand and pauses stay below 0.27 ms on the benchmarks (KNOWN_ISSUES.md, K5 and K6).

Slide 17 — What would be useful to you first?

Ask the audience and take notes. The roadmap is in docs/ROADMAP.md.